

CS Notes (Groups)

Firstname Oldroyd

July 3, 2026

2026-07-03

CS Notes (Groups)

CS Notes (Groups)

Firstname Oldroyd
July 3, 2026

Introduction

Creating Groups in Python

Permutations with SymPy

2026-07-03

└ Outline

Introduction

Creating Groups in Python

Permutations with SymPy

2026-07-03

CS Notes (Groups)
└ Introduction

Introduction

Introduction

We will use **Python** in this course to perform mathematical computations. For the first part of the computer science course, we will look at how to use Python to work with groups. We will primarily interact with Python using the **Spyder** IDE.

2026-07-03

CS Notes (Groups)

└ Introduction

└ Python

Python

We will use **Python** in this course to perform mathematical computations. For the first part of the computer science course, we will look at how to use Python to work with groups. We will primarily interact with Python using the **Spyder** IDE.

The fundamental objects we need from Python are:

- **lists**: [obj1, obj2, ...]
- **functions**: `def func(x): ...`

The fundamental objects we need from Python are:

- **lists**: [obj1, obj2, ...]
- **functions**: `def func(x): ...`

2026-07-03

CS Notes (Groups)

└─ Creating Groups in Python

Creating Groups in Python

Creating Groups in Python

A Simple Group in Python

Suppose we want to model the coins group of the previous class. Then we will need to specify the *objects* and the *actions* of the group. We can use lists to hold the objects. . .

```
# group from 1.1 has as objects arrangements of a penny and a  
↪ dime  
arrangement1 = ['penny', 'nickel']  
arrangement2 = ['nickel', 'penny']  
objects = [arrangement1, arrangement2]
```

2026-07-03

CS Notes (Groups)

└ Creating Groups in Python

└ A Simple Group in Python

A Simple Group in Python

Suppose we want to model the coins group of the previous class. Then we will need to specify the objects and the actions of the group. We can use lists to hold the objects. . .

```
# group from 1.1 has as objects arrangements of a penny and a  
↪ dime  
arrangement1 = ['penny', 'nickel']  
arrangement2 = ['nickel', 'penny']  
objects = [arrangement1, arrangement2]
```

A Simple Group in Python

...and functions to represent the actions:

```
# our action takes arrangement1 and turns it into arrangement2,  
↪ and  
# vice-versa  
def group_action(arrangement):  
    if arrangement[0] == 'penny':  
        return arrangement2  
    else:  
        return arrangement1  
  
# check that this does what we want it to  
for object in objects:  
    print(group_action(object))
```

['nickel', 'penny']

['penny', 'nickel']

2026-07-03

CS Notes (Groups)

└ Creating Groups in Python

└ A Simple Group in Python

A Simple Group in Python

```
...and functions to represent the actions:  
... and  
# our action takes arrangement1 and turns it into arrangement2,  
# vice-versa  
def group_action(arrangement):  
    if arrangement[0] == 'penny':  
        return arrangement2  
    else:  
        return arrangement1  
  
# check that this does what we want it to  
for object in objects:  
    print(group_action(object))  
  
['nickel', 'penny']  
['penny', 'nickel']
```


Example (Combining group actions using Python)

Find the result of performing the group action twice in a row using the code above.

Example (Words group again)

Implement the "three letter words" group in Python. Be careful to specify the objects and the actions!

2026-07-03

CS Notes (Groups)

└ Creating Groups in Python

└ Group Actions in Python

Example (Combining group actions using Python)

Find the result of performing the group action twice in a row using the code above.

Example (Words group again)

Implement the "three letter words" group in Python. Be careful to specify the objects and the actions!

There should be six objects and six actions that act on the objects!

Recall that a symmetry group of a geometric figure is the set of actions that preserves the shape of that figure.

Example (Symmetry group of equilateral triangle)

Implement the symmetry group of the equilateral triangle in Python.

2026-07-03

CS Notes (Groups)

└ Creating Groups in Python

└ Symmetry Groups in Python

This is more complicated than previous examples. Try working out examples by hand, then implementing in Python with a `for` loop.

Recall that a symmetry group of a geometric figure is the set of actions that preserves the shape of that figure.

Example (Symmetry group of equilateral triangle)

Implement the symmetry group of the equilateral triangle in Python.

2026-07-03

CS Notes (Groups)
└─ Permutations with SymPy

Permutations with SymPy

Permutations with SymPy

- The SymPy library lets us handle permutations in cycle notation relatively easily.
- First, we need to import useful commands as follows:

```
from sympy.combinatorics import Permutation
from sympy.combinatorics import PermutationGroup
from sympy.combinatorics import SymmetricGroup
```

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Permutations in Python

- The SymPy library lets us handle permutations in cycle notation relatively easily.
- First, we need to import useful commands as follows:

```
from sympy.combinatorics import Permutation
from sympy.combinatorics import PermutationGroup
from sympy.combinatorics import SymmetricGroup
```

- We can now create permutations as follows:

```
R = Permutation(1, 2, 3)
```

Example (Rotation followed by flip)

Let R denote clockwise rotation of the equilateral triangle by 120° and let F denote the reflection that leaves vertex 1 unchanged. Using SymPy, find $R * F$ and $F * R$.

2026-07-03

• We can now create permutations as follows:

```
R = Permutation(1, 2, 3)
```

Example (Rotation followed by flip)

Let R denote clockwise rotation of the equilateral triangle by 120° and let F denote the reflection that leaves vertex 1 unchanged. Using SymPy, find $R * F$ and $F * R$.

Example (Klein Four Group)

The symmetry group of a non-square rectangle is called the *Klein Four Group*. Create the elements of this group in Python using SymPy as above. Does the order of "multiplication" matter in this group?

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ More Computations

More Computations

Example (Klein Four Group)

The symmetry group of a non-square rectangle is called the Klein Four Group. Create the elements of this group in Python using SymPy as above. Does the order of "multiplication" matter in this group?

2026-07-03

- CS Notes (Groups)
 - └ Permutations with SymPy
 - └ Even More Computations

Even More Computations

[Example \(Cayley table\)](#)

Find the Cayley table of the symmetry group of the square

Break into four groups; each group tackles two rows of the table

Example (Cayley table)

Find the Cayley table of the symmetry group of the square

- SymPy can easily compute the parity of a permutation:

```
R.parity() # = 0 since R is even
```

0

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Parity in SymPy

Parity in SymPy

- SymPy can easily compute the parity of a permutation:

```
R.parity() # = 0 since R is even
```

0

Example (Determine existence of solution)

Consider the following 15-puzzle:

8	12	2	5
11	1	6	13
7	14	10	15
9	4	3	

Figure 1: A modified 15-puzzle

Is it possible to solve this puzzle? That is, do there exist moves that get the puzzle back into its solution state?

2026-07-03

- CS Notes (Groups)
 - └ Permutations with SymPy
 - └ Modified 15-Puzzle

Example (Determine existence of solution)

Consider the following 15-puzzle:

8	12	2	5
11	1	6	13
7	14	10	15
9	4	3	

Figure 1: A modified 15-puzzle

Is it possible to solve this puzzle? That is, do there exist moves that get the puzzle back into its solution state?

- Once we have the permutations of the Rubik's cube group, we can study them using SymPy

Example (Order of the cube group)

Find the order (that is, the size) of the Rubik's cube group.

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Studying the Rubik's Cube Group

• Once we have the permutations of the Rubik's cube group, we can study them using SymPy

Example (Order of the cube group)

Find the order (that is, the size) of the Rubik's cube group.

Example (Permutations and layers)

Explain what the following permutation does to the "cross" on the top layer of a Rubik's cube:

$$R'UF'U'.$$

Note that R' , F' and U' denote *counterclockwise* turns instead of clockwise turns.

Example (Permutations and layers)

Explain what the following permutation does to the "cross" on the top layer of a Rubik's cube.

$$R'UF'U'.$$

Note that R' , F' and U' denote *counterclockwise* turns instead of clockwise turns.

Example (Filling in corners)

Some algorithms for solving the cube focus on filling in a cross on one layer of the cube, then on filling in corners on that same layer. Explain why the following set of moves is useful for filling in one corner at a time:

$$R'D'RD$$

Hint: What does this do to the "cross" on the top layer? What about the corner cubies on the same layer?

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Permutations in the Cube Group

Example (Filling in corners)

Some algorithms for solving the cube focus on filling in a cross on one layer of the cube, then on filling in corners on that same layer. Explain why the following set of moves is useful for filling in one corner at a time:

$$R'D'RD$$

Hint: What does this do to the "cross" on the top layer? What about the corner cubies on the same layer?

Example (Finding orders of permutations)

For each of the following moves of the cube, determine how many times you would need to perform that move in a row to get back to where you started:

1. $F^2 * U$ (that is, two front clockwise turns followed by an upper clockwise turn).
2. $F * R$ (that is, a front clockwise turn followed by a right clockwise turn).

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Permutations in the Cube Group

Example (Finding orders of permutations)

For each of the following moves of the cube, determine how many times you would need to perform that move in a row to get back to where you started:

1. $F^2 * U$ (that is, two front clockwise turns followed by an upper clockwise turn).
2. $F * R$ (that is, a front clockwise turn followed by a right clockwise turn).

If you scramble a cube but keep track of the moves you used to scramble it, you can eventually solve the cube just by performing that same sequence of moves over and over.

- We can check if certain moves can be generated by other moves using the `.contains()` method:

```
G.contains(P, strict=False) # checks if permutation P is  
    ↪ in group G
```

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Moves from Other Moves

• We can check if certain moves can be generated by other moves using the `.contains()` method:

```
G.contains(P, strict=False) # checks if permutation P is  
    in group G
```

Example (Reducing the number of generators)

So far we've used six generators to construct the cube group. Show that this can also be done using five generators. *Hint*: show that a clockwise front turn can be accomplished by using some sequence of the other five turns.

2026-07-03

CS Notes (Groups)

└ Permutations with SymPy

└ Moves from Other Moves

Example (Reducing the number of generators)

So far we've used six generators to construct the cube group. Show that this can also be done using five generators. *Hint*: show that a clockwise front turn can be accomplished by using some sequence of the other five turns.

This can be done with the `contains` method on a "subgroup" built out of the other five motions.